

Object Classes

- This class is in java.lang package.
 - It is parent class of all classes in java, it means every class in java defaultly inherits from object class.
 - Object class me hum common behaviours rkhte ha jo hum har class ke liye commonly zaruri ha.
 - Object class ka reference kisi bhi class ko or unke object ko point kr skta ha, because of polymorphism.
-

Core methods of object class:

1. **toString():** converts anything to String.

```
public class toStringMethod {  
    public static void main(String[] args) {  
        //Practicing toString method-->  
        Student s1 = new Student();  
        s1.name = "Priyansh";  
        s1.age = 24;  
        System.out.println(s1.toString()); // agr me sirf println(s1) bhi likhu tb bhi ye call ho jayga.  
        System.out.println(s1); // both means same. result will be same.  
    }  
}  
  
class Student{ // every class defaultly extends Object class.  
    String name;  
    int age;  
    @Override  
    public String toString(){  
        return (name + ", " + age);  
    }  
}
```

2. **equals():** compare 2 objects & returns true/false (boolean value).

```
public class equalsMethod {  
    public static void main(String[] args) {  
  
        // practicing equals Method shows boolean value.  
        Student s1 = new Student();  
        s1.name = "Priyansh Sharma";  
        s1.age = 23;  
        Student s2 = new Student();  
        s2.name = "Priyansh Sharma";  
        s2.age = 23;  
        System.out.println(s1.equals(s2)); // internally--> Object obj = new Strudent();  
    }  
}  
  
class Student{ // every class defaultly extends Object class.  
    String name;  
    int age;  
    @Override  
    public boolean equals(Object obj){  
        if(this == obj) {  
            return true;  
        }  
        if (obj == null){
```

```

        return false;
    }
    //check if both classes are of type Student.
    if(obj.getClass() != this.getClass()){
        return false;
    }
    Student s = (Student) obj; // typecast obj to student, therefore s2 is now
student
    return (this.name == s.name && this.age == s.age);
}
}

```

3. **hashCode():** Return an integer in hexadecimal format of an Object.

Rules of hashCode():

- **If 2 objects are equal than their hasCode must be equal.**
- **But the reverse might not be true.** It means if hashCode of 2 different objects are equal it doesn't means the objects are equal.

```

import java.util.Objects;
public class hashCodeMethod {
    public static void main(String[] args) {
        // practicing hashCode Method shows boolean value.
        Student s1 = new Student();
        s1.name = "Ram Chauhan";
        s1.age = 25;
        Student s2 = new Student();
        s2.name = "Ram Chauhan";
        s2.age = 25;
        System.out.println(s1.equals(s2)); // internally--> Object obj = new Strudent();
        System.out.println(s1.hashCode() == s2.hashCode()); // therefore hashCode are
equal--> True
        System.out.println(s1.hashCode()); // to check s1 hashCode.
        System.out.println(s2.hashCode()); // to check s2 hashCode.
    }
}
class Student{ // every class defaultly extends Object class.
    String name;
    int age;
    @Override
    public boolean equals(Object obj){
        if(this == obj) {
            return true;
        }
        if (obj == null){
            return false;
        }
        //check if both classes are of type Student.
        if(obj.getClass() != this.getClass()){
            return false;
        }
        Student s = (Student) obj; // typecast obj to student, therefore s2 is now
student's object.
        return (this.name == s.name && this.age == s.age);
    }
}

```

```

@Override
public int hashCode(){
    return Objects.hash(name, age); // here "Objects" is another class than Object
class
} // "Objects" class java.util package me ha.
// to isko uper import krna hoga.
}

```

4. **getClass():** Return runtime class of an object.

- **Instance Operator** is also covered below.

```

public class getClass {
    public static void main(String[] args) {
        //practicing getClass Method
        Animal a = new Animal();
        Animal d = new Dog();
        System.out.println(a.getClass().getName()); // Ans-->Animal. Runtime class animal
ha.

        System.out.println(d.getClass().getName()); // Ans-->Dog. Runtime class Dog ha.
        //Practicing "instance Operator"--> if an object is "instance" of class or any oof
its sub-class.

        System.out.println(a instanceof Object); //true
        System.out.println(a instanceof Animal); //True
        System.out.println(a instanceof Dog); //False
        System.out.println(d instanceof Animal); //True
    }
}

class Animal{
}

class Dog extends Animal{
}

```

Cloning method:

1. **clone():** It helps to create copy of an object.

- This method is protected type in Object class.
- To **@Override** this clone method in any class, that class should be “clonable interface”.
- Not every object should be Cloneable, therefore it uses “clonable interface”.
- Cloneable interface is empty, these type of interfaces are known as **MARKER INTERFACE**.

```

public class cloneMethod {
    public static void main(String[] args) throws CloneNotSupportedException{
        Student s1 = new Student();
        s1.name = "Priyansh";
        s1.age = 23;

        Student s2 = (Student) s1.clone(); //Typecaste to Student class from object class.
        System.out.println(s2.name); // Priyansh
        System.out.println(s2.age); // 23
    }
}

class Student extends Object implements Cloneable { // to override clone class should be
Cloneable "interface"
}

```

```
String name;  
int age;  
  
@Override  
protected Object clone() throws CloneNotSupportedException{//it is protected.  
    return super.clone();// 'Super' to access parent class variable.  
}  
}
```

Garbage collection method:

1. `finalize()`; It is not used now. It is deprecated now.

Threads methods:

1. `wait()`;
2. `notify()`;
3. `notifyAll()`;